

Informe de FRONTED 24/03/2026

-PROYECTO ATALANTA

Informe de Arquitectura y Seguridad Frontend (Client-Side)

Proyecto: Atalanta Web & Gestor de Incidencias

Rol: Arquitecto de Software Frontend Senior / Especialista WebSec

Fecha: 24 de Marzo de 2026

1. Arquitectura de la Interfaz y Flujo de Navegación

Estructura Documental y Vistas

El proyecto frontend está parcelado en dos dominios visuales principales servidos estáticamente (MPA - Multi-Page Application):

1. Landing Page (

inicio/atalanta.html): Portal de marketing estático con animaciones CSS/JS basadas en iteración cronometrada, manejo del visor de imágenes con layouts estáticos.

2. Aplicación Web (gestor-de-incidencias1/):

- **Módulo de Autenticación (**

login.html,

login.js): Interfaz dual interactiva (Login/Registro) orquestada mediante alternancia de clases CSS (.active) simulando pestañas, y un sistema en "pasos" asíncronos para el asistente de registro.

- **Dashboards (**

admin.html,

trabajador.html): Interfaces de gestión ancladas sobre la carga estática.

Gestión del Estado y Mantenimiento de Autenticación

- **Ausencia de Framework Persistente:** No existe un inyector VDOM tipo React/Vue. La persistencia se delega enteramente a @supabase/supabase-js, el cual interactúa silenciosamente con el localStorage interceptando la expiración del JWT de Auth.
- **Firewall Visual (Guards de Ruta):** Al acceder a áreas restringidas, la primera línea es la promesa await supabase.auth.getSession(). Este proceso determina instantáneamente si hay token válido; si es nulo, un bloqueo "Redirect" aborta la carga disparando al usuario de vuelta a ../login.html.

Componentes Dinámicos

- Renderización manual utilizando **Template Literals** inyectados iterativamente sobre nodos iterables (<tbody>, <div>). El proceso actual actualiza en bloque el innerHTML re-dibujando los componentes tras hidratar los JSON que devuelve la capa Data.

2. Comunicación con el Backend y APIs

Patrón Cliente SDK (Supabase)

Cada vista interactiva importa el cliente oficial mediante CDN ESM modular: import { createClient } from 'https://cdn.jsdelivr.net/npm/@supabase/supabase-js/+esm';

- El patrón de inicialización asume un modelo Singleton repetitivo por cada vista. Las consultas fluyen en lenguaje PostgREST subyacente.

- Uso sistemático de encadenamiento `.from(x).select(x)`, `.eq()`, indicando que la gran fuerza del aplicativo está en el conocimiento exhaustivo de la estructura relacional subyacente que opera post-token.

Dependencias Externas (Integraciones 3rd-Party)

- **EmailJS (**

`atalanta.js)`: El cliente usa un script asíncrono pasándole una *Public Key* estática expuesta. Intercepta eventos DOM (`Type="submit"`) para disparar la carga Petición-SMTP a los correos sin backend intermediario.

3. Auditoría de Seguridad en el Cliente (Client-Side WebSec)

Protección y Neutralización de XSS (Cross-Site Scripting)

- **Status Actual:** Seguro / Completamente Mitigado en Core Operations.
- **Vector Bloqueado:** Históricamente, renderizar variables como `${u.nombre}` directo en variables DOM asume entrada limpia. Ahora, el código posee el método `escapeHTML` como interceptor global previo a todas las interpolaciones, transformando (`< > & " '`) a su notación en entidades seguras. El Stored XSS en el Panel Dashboard ha sido paralizado eficazmente.

Manejo de Storage y Sesiones Duras

- **Vía de Almacenamiento:** JWT gestionado localmente por API `localStorage`.
- **Análisis Crítico:** Almacenar tokens masivos en Storage asume un riesgo inmenso **si** se permitieran inyecciones DOM (XSS). Sin embargo, habiéndose taponado todas las

grietas críticas (ver arriba), un cibercriminal ha perdido su palanca central para la extracción estática del JWT local, rindiendo la capa frontal robusta.

Firewall de Validación Lógica (Lado Cliente)

- **Registro (**

login.js): La validación estricta obliga a superar Regex sólidas para correo y rechaza automáticamente strings minúsculas o vacías de contraseñas garantizando entropías mayores a 8 dígitos con evaluación en tiempo real (Barra de Fortaleza). Ahorra miles de transacciones basura al servidor.

4. Calidad del Código y Mantenimiento

Calidad de Excepciones y Telemetría

- La refactorización actual ha pulido la exposición explícita (stack-traces filtrados en producción). El sistema ahora hace uso del silencioso pero auditable `console.error()` ante anomalías. Anteriormente, un mal uso de invocaciones generaba fugas analíticas.

Métricas Visuales y Rendimiento Analizado

- **Redundancia CDN (Cuello de red):** Utilizar módulos de origen Remoto `https://cdn...` en cada uno de los tableros HTML impone renegociaciones de red continuas por falta de Caché Unificada (Bundle), limitando el FCP (First Contentful Paint).
 - **Cargas de Artefactos Pesados:** La renderización recurrente sobre el nodo padre completo (reiniciar nodos de cero para cada modificación +=) genera repintados intensivos DOM "layout thrashing".
-

Recomendaciones Tecnológicas (Roadmap FASE 2.5)

1. **Implementar "Bundle Architecture" (Vite o Webpack):** La importación descentralizada CDN genera demoras de lectura/red. Al empaquetar Supabase localmente, los JS de login y dashboard cargarían de modo ultrarápido desde discos encriptados (minificados).
2. **Transición a Nodos DOM (createElement / textContent):** A pesar del escudo escapeHTML, se recomienda la migración definitiva a la creación estructurada JS (document.createElement) para manipular la tabla de usuarios, anulando para siempre la superficie dependiente de innerHTML.
3. **Restricción EmailJS / CSP Headers:** Configurar CSP (*Content-Security-Policy*) limitando quién llama y a donde van los POST externos. Blindar dominios para EmailJS en su interfaz impidiendo que cuentas piratas llamen mediante Postman usando la API expuesta en Frontend.

JL